.UBAECONÓMICAS

Tecnicatura en Gestión y Análisis de Datos

TALLER DE PROGRAMACIÓN PARA EL ANÁLISIS DE DATOS

U1 - Clase 1

Introducción a la programación

Contenidos de la clase

01

Conceptos Generales de Informática

Software, Sistemas Operativos, Lenguajes de Programación

02

Introducción a la Programación

Variables, Tipos, Comentarios, Bucles, Condiciones, Funciones

03

Diagramas de Flujo y Ejercicios

Algoritmos, Diagramas, Pseudocódigo, Actividad práctica

01

Conceptos Generales de Informática

Repaso de fundamentos esenciales de informática

.UBAECONÓMICAS

Hardware

DEFINICIÓN

El hardware son los componentes físicos y tangibles de un sistema informático (CPU, memoria, almacenamiento, periféricos) que procesan datos y ejecutan instrucciones del software.

Software

DEFINICIÓN

Conjunto de instrucciones de computadora que le indica cómo realizar tareas.

El software es el componente lógico e intangible de un sistema informático. Sin él, el hardware no puede funcionar.

Software — Tipos y Ejemplos

Software de Aplicaciones

- Procesadores de texto (Word)
- Planillas de cálculo (Excel)
- Juegos
- Aplicaciones móviles
- Navegadores web

Software del Sistema

- Sistemas operativos (Windows, Linux)
- Bases de datos (MySQL, Oracle)
- Compiladores e intérpretes
- Drivers de hardware
- Utilidades del sistema

Sistemas Operativos

Software que controla y dirige la operación de una computadora, facilitando su uso al usuario.

Software que gestiona los recursos de hardware y provee servicios a los programas de aplicación.

Aplicaciones

Word, Chrome, Juegos...

Sistema Operativo

Windows, Linux, iOS...

Hardware

CPU, RAM, Disco...

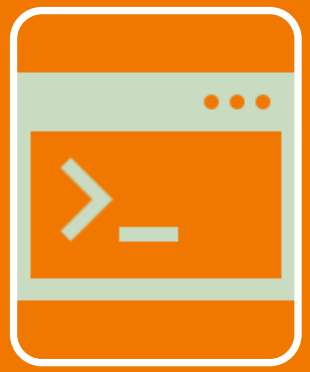
Sistemas Operativos — Ejemplos

Windows	iOS / macOS	Linux	Android	Tizen
Microsoft	*Apple*	*Open Source*	*Google*	*Samsung*

Cada dispositivo tiene su propio sistema operativo. Smartphones utilizan Android o iOS, computadoras de escritorio usan Windows, macOS o Linux, y algunos dispositivos inteligentes emplean Tizen u otros sistemas especializados.

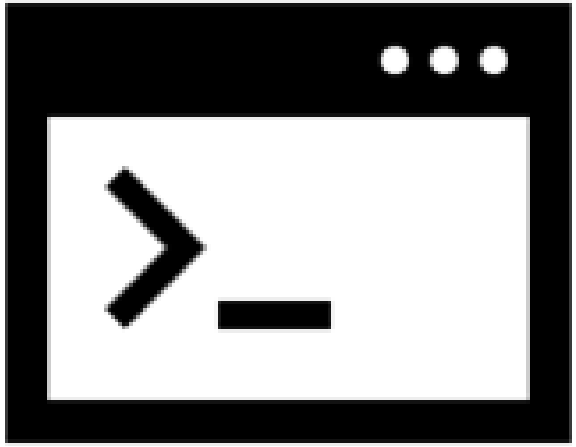
Las computadoras

deben seguir las instrucciones paso a paso para hacer las cosas.



Las computadoras

necesitan instrucciones muy específicas para hacer las cosas



El tipo de instrucciones que necesitan las computadoras se puede comparar con aquellas que recibe un ser humano cuando quiere aprender a hacer una nueva tarea.

A diferencia del ser humano, las computadoras siempre necesitan las instrucciones completas y detalladas.

En cambio, el ser humano puede comprender el problema por sus propios medios y realizar una tarea sin un detalle exhaustivo.

02

Introducción a la Programación

Conceptos básicos del desarrollo de software

.UBAECONÓMICAS

Lenguajes de Programación

DEFINICIÓN

Lenguaje que proporciona instrucciones que permiten a un programador escribir operaciones y algoritmos con el fin de controlar el funcionamiento de una computadora.

(Código Fuente)

Código Fuente

Lo que escribe el programador

→

Compilador / Intérprete

Traduce a lenguaje máquina

→

Lenguaje Máquina

0s y 1s que ejecutará la CPU

Algoritmo

DEFINICIÓN

Conjunto definido de operaciones o pasos ordenados que permiten realizar una tarea.

Los algoritmos son independientes del lenguaje de programación y pueden expresarse en pseudocódigo, diagramas de flujo o código real.

1. **Definido**
   Cada paso está claramente especificado
2. **Finito**
   Termina en un número finito de pasos
3. **Ordenado**
   Los pasos siguen una secuencia lógica

Entorno de Desarrollo

DEFINICIÓN

Conjunto de herramientas, software y configuraciones que los programadores utilizan para escribir, depurar, probar y modificar código fuente de aplicaciones

Muchos pueden autocompletar el código o resaltar donde pueda haber errores.

IDE (Entorno de Desarrollo Integrado): VS Code, PyCharm, Eclipse

Paradigmas de Desarrollo

DEFINICIÓN

Enfoque o estilo de programación que define cómo se organiza y estructura el código para resolver problemas.

Cada paradigma propone una forma distinta de pensar la solución.

Python es multiparadigma

Combina los tres enfoques según la necesidad. En data science se usa principalmente el estilo procedimental y POO.

Imperativo / Procedimental

El programa es una secuencia de instrucciones que la computadora ejecuta paso a paso.

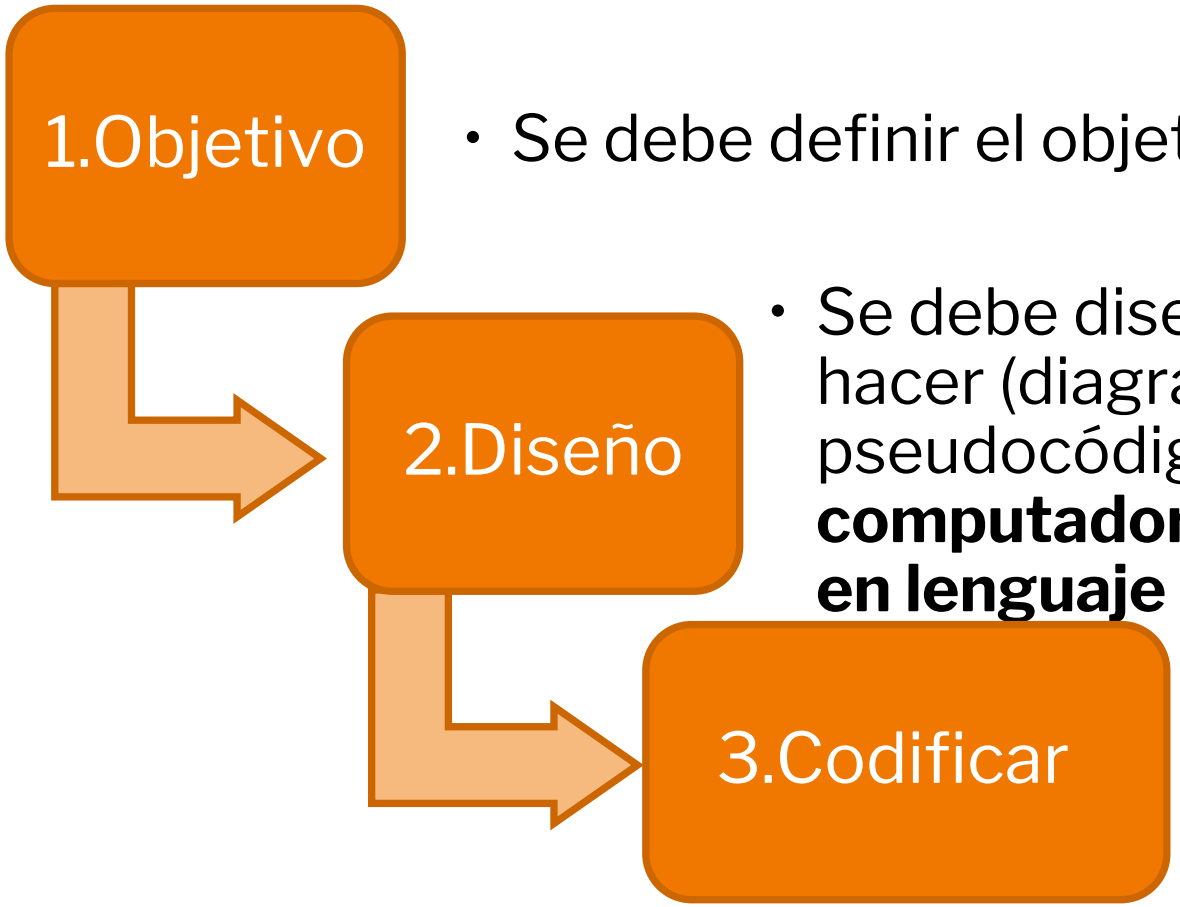
Orientado a Objetos (POO)

Organiza el código en objetos que combinan datos y comportamiento. Facilita la reutilización.

Funcional

Trata la computación como evaluación de funciones matemáticas.

Proceso de escritura del código



- Se debe definir el objetivo
- Se debe diseñar lo que necesita hacer (diagrama de flujo, pseudocódigo (**es lo que la computadora debe hacer, escrito en lenguaje natural.)**, etc.)
- Se debe codificar cada paso

Variables

DEFINICIÓN

Posición de memoria de la computadora a la cual se le asigna un nombre.

Las variables tienen diferentes tipos de datos, lo que ayuda a la computadora a saber qué se puede hacer con ellas (por ejemplo, los valores numéricos se pueden sumar, los textos no)

Algunos tipos de datos

Símbolo
Números enteros
Números decimales (punto flotante)
Cadenas de caracteres
Booleanos (verdadero – falso)
Listas

Operadores aritméticos, de comparación, lógicos

Los operadores son símbolos que manipulan, comparan o asignan valores almacenados en variables (espacios de memoria), permitiendo realizar acciones como cálculos, comparaciones o toma de decisiones.

Variables

Tipos de lenguajes en función de como verifican los tipos de las variables

Tipado estático

Los tipos se verifican en tiempo de compilación, antes de ejecutar.

Si hay un error de tipos, el programa directamente no compila.

Ejemplo: Java, C, Go.

Tipado dinámico

Los tipos se verifican en tiempo de ejecución. No hace falta declararlos en el programa.

El programa puede fallar en medio de la ejecución si hay un error de tipos.

Ejemplo: **Python**, JavaScript, Ruby.

Sentencias condicionales

DEFINICIÓN

Estructura de programación que nos permite tomar decisiones en tiempo de ejecución. En función del resultado de una pregunta el programa realiza una acción u otra

Ciclos

DEFINICIÓN

Estructura de programación que nos permite repetir la misma porción de código

Funciones

DEFINICIÓN

Estructura de programación que nos permite poner nombre a un bloque de código, el cual podemos reutilizar en el programa.

Estas estructuras pueden recibir valores para trabajar (parámetros) y devolver un resultado

Bibliotecas - Liberías

DEFINICIÓN

Las librerías (o bibliotecas) en programación son conjuntos de archivos de código preescrito, y reutilizable que permiten a los desarrolladores solucionar tareas comunes sin escribir todo desde cero.

¿Cómo arrancamos a pensar los algoritmos?

Dos herramientas para estructurar ideas:

{ } Pseudocódigo

Descripción informal con lenguaje natural cercano al código

◆ Diagrama de Flujo

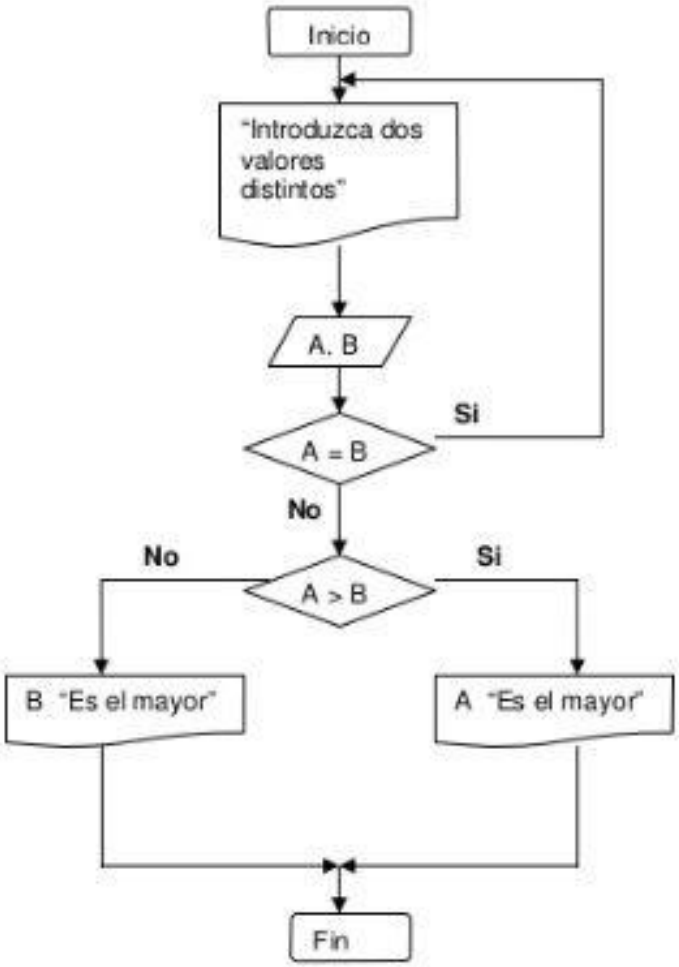
Representación gráfica con figuras geométricas estandarizadas

Pseudocódigo

HAHAHAHAHAHA

Diagrama de flujo

- Un diagrama de flujo es una representación visual de un proceso donde se detallan tareas y decisiones.
- Su finalidad es expresar de manera simple y clara lo que ocurre en un programa, de manera que sea fácil de comprender.
- Ejemplo: Hacer un diagrama de flujo que permita leer 2 números diferentes y nos diga cual es el mayor de los 2 números.



Elementos de un diagrama de flujo

Símbolo	Significado
	Inicio o Término: indica el principio o fin de un flujo. Puede denotar una acción o un lugar. Además, se usa para indicar una unidad administrativa o persona que recibe o proporciona información.
	Actividad: describe las funciones que desempeñan las personas involucradas en el procedimiento
	Documento. Represente cualquier documento que entre, se utilice, se genere o salga del procedimiento
	Decisión o Alternativa: indica un punto dentro del flujo en donde se debe tomar una decisión entre dos o más opciones.
	Almacenar datos: indica la acción del almacenamiento de datos en una tabla o archivo de datos.
	Base de datos: indica la existencia de un conjunto de tablas con datos almacenada previamente.
	Conector de página: representa una conexión o enlace con otra hoja diferente en la que continúa el diagrama de flujo.
	Conector: representa una conexión o enlace de una parte del diagrama de flujo con otra parte del mismo

03

Ejercicios Prácticos

Aplicamos los conceptos con pseudocódigo y diagramas

Ejercicio 1

Enunciado

Pedirle al usuario la base y la altura de un rectángulo, calcular su área e imprimirla

💭 *Intentá resolverlo antes de ver la solución en la siguiente diapositiva*

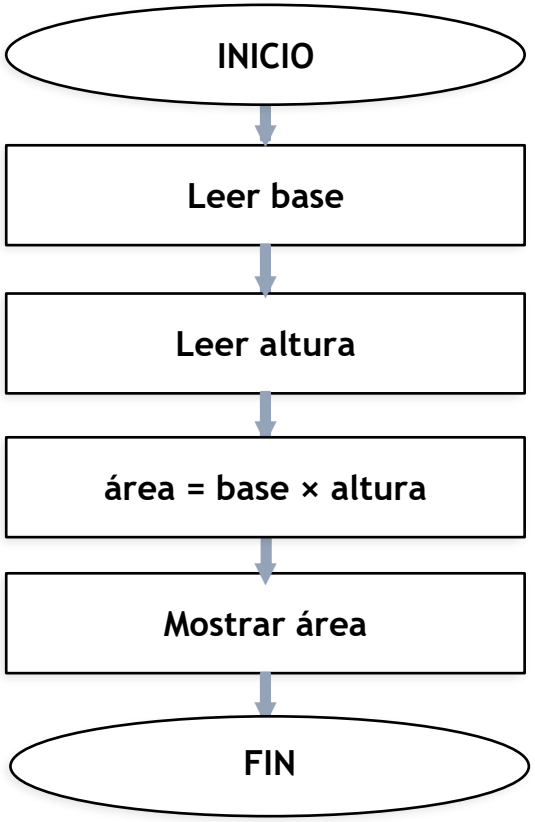
Ejercicio 1

Enunciado

Pedirle al usuario la base y la altura de un rectángulo, calcular su área e imprimirla.

Estructura utilizada:

- Entrada: base, altura
- Proceso: área = base × altura
- Salida: área



Ejercicio 2

Enunciado

Pedirle al usuario que ingrese un número entero.

Mostrar un mensaje indicando si el número ingresado es positivo, negativo o cero.

Intentá resolverlo antes de ver la solución en la siguiente diapositiva

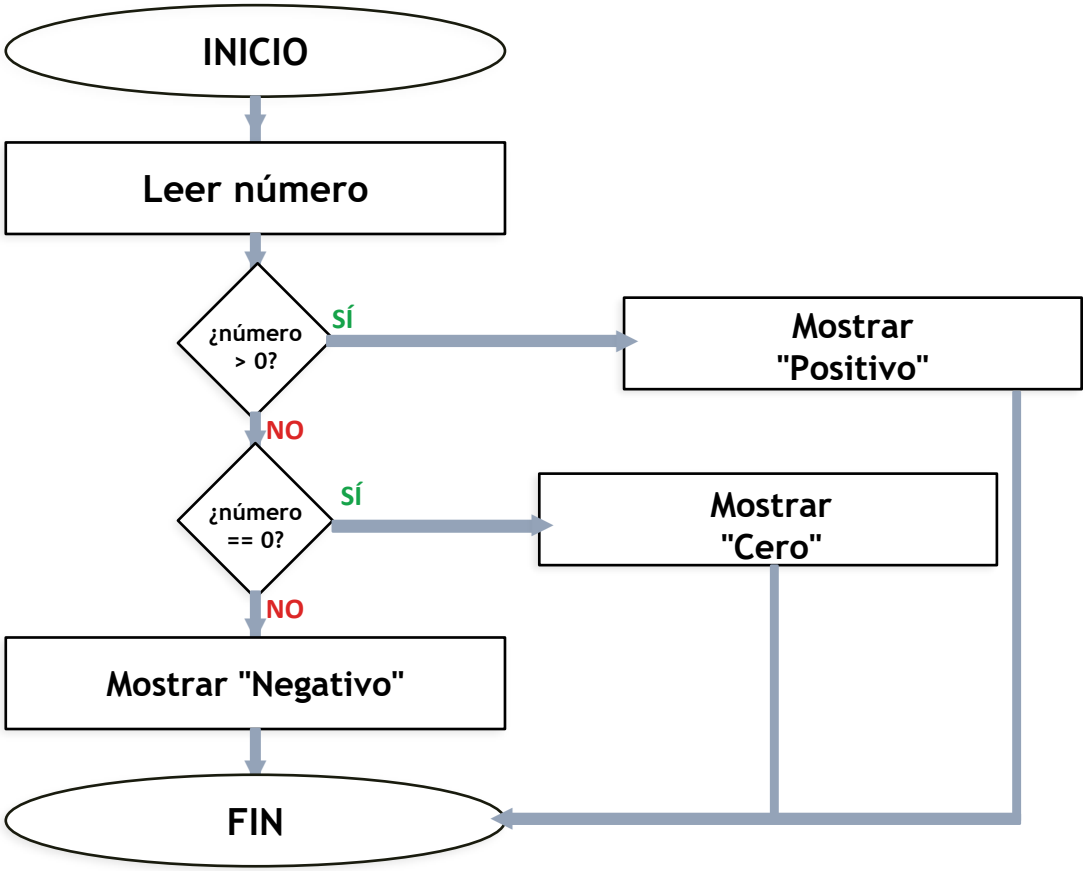
Ejercicio 2

Enunciado

Pedir un número al usuario y mostrar un mensaje indicando si es positivo, negativo o cero.

Estructura utilizada:

- Secuencia: leer número
- Decisión: ¿número > 0?
- Rama Sí → 'Positivo'
- Rama No → ¿número == 0?
- Sí → 'Cero' / No → 'Negativo'



Ejercicio 3

Enunciado

Pedir números al usuario uno por uno y acumular la suma.

Cuando el usuario ingrese 0, mostrar el total acumulado y terminar.

💭 *Intentá resolverlo antes de ver la solución en la siguiente diapositiva*

Ejercicio 3

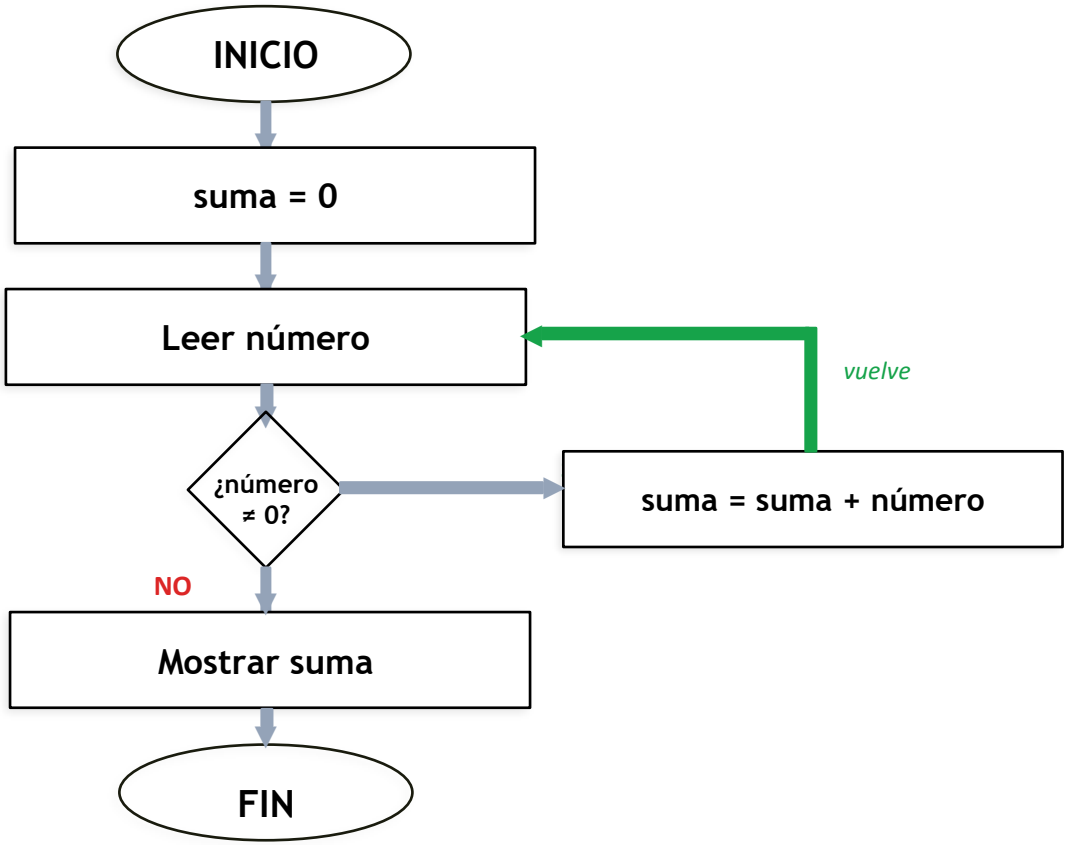
Enunciado

Pedir números al usuario uno por uno y acumular la suma. Cuando el usuario ingrese 0, mostrar el total y terminar.

Estructura:

- Inicializar suma = 0
- Leer número
- Mientras número ≠ 0: sumar y volver a leer
- Al salir del ciclo: mostrar suma

↺ Ciclo controlado por variable (valor 0)



Ejercicio 4

Enunciado

Pedir un número entero positivo N al usuario.

Calcular su factorial: $N! = 1 \times 2 \times 3 \times \ldots \times N$.

Mostrar el resultado

Intentá resolverlo antes de ver la solución en la siguiente diapositiva

Ejercicio 4

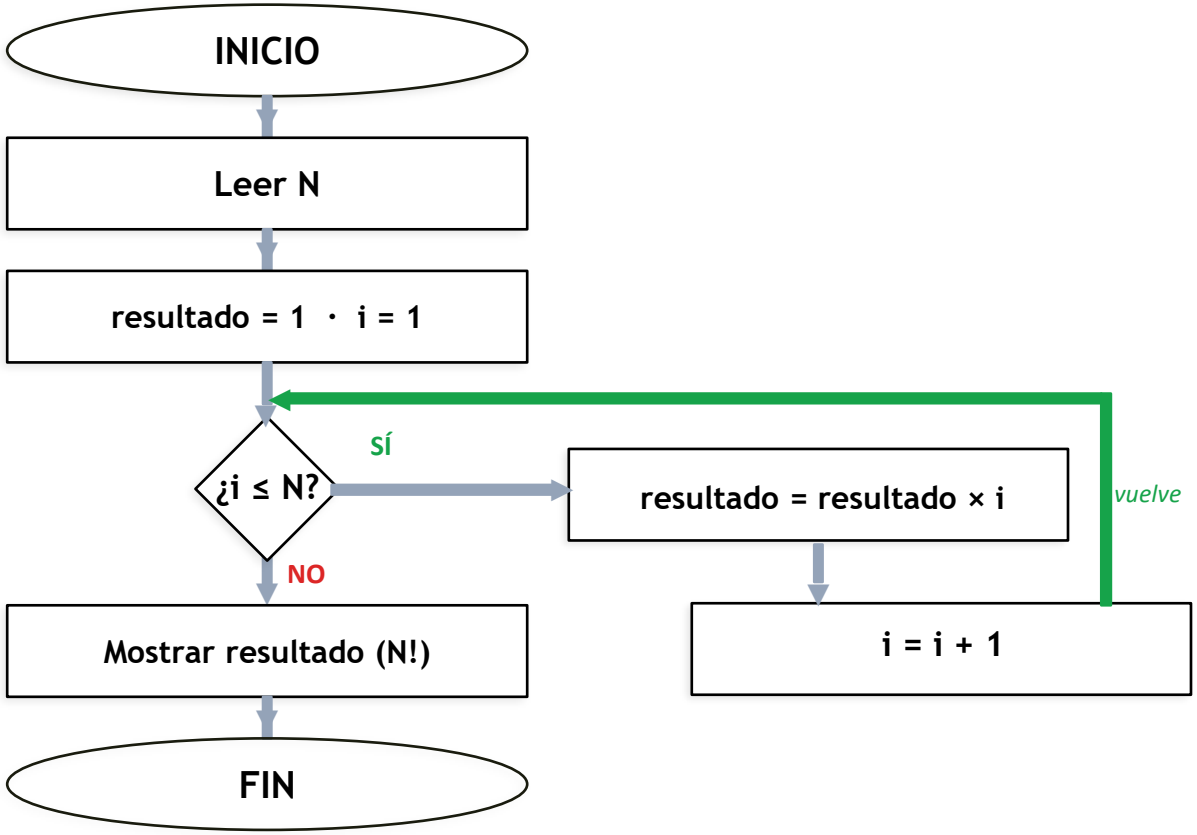
Enunciado

Pedir un número entero positivo N al usuario y calcular su factorial (N! = 1 × 2 × 3 × … × N).

Estructura:

- Leer N
- resultado = 1 · i = 1
- Mientras i ≤ N: resultado = resultado × i
- i = i + 1 → volver al ciclo
- Mostrar resultado (= N!)

↺ Ciclo acumulador de producto



Actividad 1

Enunciado

Crear el diagrama de flujo que permita determinar si un año ingresado es bisiesto.

Un año es bisiesto si:

- Es divisible por 4 → podría ser bisiesto

  EXCEPTO si es divisible por 100 → NO es bisiesto

  EXCEPTO si es divisible por 400 → SÍ es bisiesto

Ejemplos bisiestos: 2000, 2004, 2024.

Ejemplos no bisiestos: 1900, 2100, 2023

Actividad 2

Enunciado

Crear el diagrama de flujo que permita determinar si un número ingresado por el usuario es primo.

Un número es primo si:

- Es entero y estrictamente mayor que 1.
- No es divisible por ningún número entre 2 y N-1.

Ejemplos de primos: 2, 3, 5, 7, 11, 13...

Ejemplos de no primos: 4, 6, 9, 12...

.UBAECONÓMICAS

Tecnicatura en Gestión y Análisis de Datos

¡MUCHAS GRACIAS POR SU ATENCIÓN!